

# Energy Efficiency in Railway Scheduling

Paul Tennigkeit, Georgi Abshilava

Universität Potsdam

21.09.2026

# Outline

Motivation: Energy Efficiency Problem

Baseline ASP Model

Energy Encoding Mechanics

Results and Comparison with Baseline Model

Conclusion

# Motivation: Energy Efficiency Problem

# Baseline ASP Model

- ▶ Establishes rules to handle essential train behavior
  - ▶ Directions
  - ▶ Movements
  - ▶ Transitions
  - ▶ Collision and Reachability Constraints
  - ▶ Schedule Optimization

# Directions and Movements

```
% Directions
```

```
dir(s).
```

```
dir(n).
```

```
dir(w).
```

```
dir(e).
```

```
% Movement Operation
```

```
moving(move_forward).
```

```
moving(move_left).
```

```
moving(move_right).
```

```
% Movement on Grid
```

```
delta(A,s,1,0) :- moving(A).
```

```
delta(A,n,-1,0) :- moving(A).
```

```
delta(A,e,0,1) :- moving(A).
```

```
delta(A,w,0,-1) :- moving(A).
```

# Transitions

```
% Transition facts: To represent different track types
% ID : track ID
% Dir1: pointing direction of agent
% Act : associated action with movement towards exit
% Dir2: exit direction
transition(ID, Dir1, Act, Dir2).
```

# Collision Constraints

```
% Trains can not occupy the same cell at the same time
:- stay(ID1,X,Y,T),
   stay(ID2,X,Y,T),
   ID1 < ID2.
```

```
% Trains can not swap their position
:- stay(ID1,X1,Y1,T),
   stay(ID2,X2,Y2,T),
   stay(ID1,X2,Y2,T+1),
   stay(ID2,X1,Y1,T+1),
   ID1 < ID2.
```

# Reachability Constraints

```
% define states (cell, time, direction),  
% which are reachable by a train  
reachable(ID,T,A,X2,Y2,T2,Dir2) :-  
    A != wait,  
    position(ID,(X,Y),T,Dir1),  
    cell((X,Y),Cell),  
    transition(Cell,Dir1,A,Dir2),  
    delta(A,Dir2,DX,DY),  
    X2 = X + DX,  
    Y2 = Y + DY,  
    speed(ID,S),  
    T2 = T + S,  
    time(T2).  
  
reachable(ID,T,wait,X,Y,T+1,Dir) :-  
    position(ID,(X,Y),T,Dir),  
    time(T+1).
```

# Reachability Constraints

```
% Choose exactly one valid transition at each time step
1 { position(ID,(X2,Y2),T2,Dir2)
    : reachable(ID,T,A,X2,Y2,T2,Dir2)
} 1 :- position(ID,(_,_),T,_),
    not reached(ID,T).

% Track the time interval a train spends inside a cell.
% This is to make enforcing collision constraints easier.
chosen(ID,T,A,X,Y,X2,Y2,T2) :-
    position(ID,(X,Y),T,_),
    position(ID,(X2,Y2),T2,_),
    reachable(ID,T,A,X2,Y2,T2,_).

% a train reached its target if their position matches
reached(ID,T) :- position(ID,(X,Y),T,_),
    end(ID,(X,Y),_).
```

# Schedule Optimization

```
% stop searching if train is not
% in time to reach its target anymore
:- position(ID,(X,Y),T,_),
   end(ID,(EX,EY),Arr),
   speed(ID,InvSpd),
    $T + (|X-EX| + |Y-EY|)*InvSpd > Arr.$ 

% the arrival times should be minimized
reached_earlier(ID,T) :- reached(ID,T),
                           reached(ID,T0),  $T0 < T.$ 
arrival(ID,T) :- reached(ID,T),
                  not reached_earlier(ID,T).

% minimize summed arrival times
#minimize { T@2,ID : arrival(ID,T) }.
```

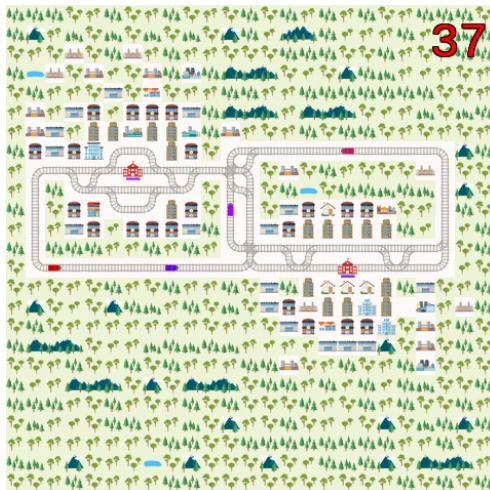


Figure: Baseline Encoding in Action (at timestep 37) with train 0, train 1, train 2 and train 3

Table: Train 0

| Time step | Position | direction | status | Given Command |
|-----------|----------|-----------|--------|---------------|
| 34        | (11, 1)  | s         | moving | move_forward  |
| 35        | (12, 1)  | s         | moving | move_forward  |
| 36        | (13, 1)  | s         | moving | move_forward  |
| 37        | (13, 2)  | e         | moving | move_forward  |
| 38        | (13, 3)  | e         | moving | move_forward  |
| 39        | (13, 4)  | e         | moving | move_forward  |
| 40        | (13, 5)  | e         | moving | move_forward  |

Table: Train 1

| Time step | Position | direction | status | Given Command |
|-----------|----------|-----------|--------|---------------|
| 34        | (7, 20)  | w         | moving | move_forward  |
| 35        | (7, 19)  | w         | moving | move_forward  |
| 36        | (7, 18)  | w         | moving | move_forward  |
| 37        | (7, 17)  | w         | moving | move_forward  |
| 38        | (7, 16)  | w         | moving | move_forward  |
| 39        | (7, 15)  | w         | moving | move_forward  |
| 40        | (7, 14)  | w         | moving | move_forward  |

Table: Train 2

| Time step | Position | direction | status  | Given Command |
|-----------|----------|-----------|---------|---------------|
| 28        | (12, 11) | w         | moving  | move_forward  |
| 32        | (11, 11) | n         | moving  | move_forward  |
| 36        | (10, 11) | n         | moving  | wait          |
| 37        | (10, 11) | n         | stopped | move_forward  |
| 41        | (9, 11)  | n         | moving  | move_forward  |
| 45        | (8, 11)  | n         | moving  | move_left     |
| 49        | (8, 10)  | w         | moving  | move_forward  |

Table: Train 3

| Time step | Position | direction | status | Given Command |
|-----------|----------|-----------|--------|---------------|
| 34        | (13, 5)  | e         | moving | move_forward  |
| 35        | (13, 6)  | e         | moving | move_forward  |
| 36        | (13, 7)  | e         | moving | move_forward  |
| 37        | (13, 8)  | e         | moving | move_forward  |
| 38        | (13, 9)  | e         | moving | move_forward  |
| 39        | (13, 10) | e         | moving | move_forward  |
| 40        | (13, 11) | e         | moving | move_forward  |

# Energy Encoding Mechanics

- ▶ Energy Cost Factors and Power Types
- ▶ Energy Consumption for Different Types of Actions (Braking, Turning, Coasting)
- ▶ Encoding Optimization

## Cost Factors and Power Types

```
% Energy cost factor of trains
% based on power type + weight
pow_cost_fac(ID,1) :- pow_type(ID, electro).
pow_cost_fac(ID,2) :- pow_type(ID, hybrid).
pow_cost_fac(ID,3) :- pow_type(ID, diesel).
pow_cost_fac(ID,4) :- pow_type(ID, hydrogen).

% A train's weight is defined in
% the environment's logic program
cost_fac(ID,CF) :- pow_cost_fac(ID,PCF),
                    weight(ID,W),
                    CF = PCF * W.
```

## Cost of Braking

```
% Tracks whether the train was waiting
% in the previous time step
prev_wait(ID,T) :- action(train(ID),_,T),
                    time(T-1),
                    action(train(ID),wait,T-1).

energy(ID,T+1,E) :- action(train(ID),wait,T),
                    speed(ID,S),
                    not prev_wait(ID,T),
                    cost_fac(ID,CF),
                    E = CF * 24 / S.
```

# Cost of Coasting

```
energy(ID,T2,E) :- action(train(ID),A,T),  
                    A != wait,  
                    position(ID,_,T,Dir),  
                    speed(ID,S),  
                    T2 = T + S,  
                    position(ID,_,T2,Dir),  
                    cost_fac(ID,CF),  
                    E = CF.
```

## Cost of Braking

```
energy(ID,T2,E) :- position(ID,_,T,Dir1),  
                    speed(ID,S),  
                    T2 = T + S,  
                    position(ID,_,T2,Dir2),  
                    Dir1 != Dir2,  
                    cost_fac(ID,CF),  
                    E = CF * 12 / S.
```

# Encoding Optimization

```
% Optimize for energy efficiency  
% Arrival time still has higher priority than energy  
#minimize { E@1,ID,T : energy(ID,T,E) }.
```

# Results and Comparison with Baseline Model

# Conclusion